

|                         |                                    |
|-------------------------|------------------------------------|
| <b>Document Number:</b> | <b>P0911R1</b>                     |
| Date:                   | 2018-03-15                         |
| Project:                | Programming Language C++, CWG, LWG |
| Revises:                | P0911R0                            |
| Reply to:               | gorn@microsoft.com                 |

# Rebase the Coroutines TS onto the C++17 Standard

---

## Introduction

The current working draft for the Coroutines TS is based on C++14. This paper proposes revising the draft to reflect the new standard.

## Wording updates

### Clause renumbering

Due to new ISO guidelines, C++17 has changed the numbers of nearly all the clauses relative to C++14. In Coroutines TS replace all clause numbers and references to stable names from C++14 based to C++17 based as follows:

| <b>Stable Name</b>             | <b>C++14 based numbering</b> | <b>C++17 based numbering</b> |
|--------------------------------|------------------------------|------------------------------|
| atomics                        | 29                           | 32                           |
| basic                          | 3                            | 6                            |
| basic.link                     | 3.5                          | 6.5                          |
| basic.lookup.classref          | 3.4.5                        | 6.4.5                        |
| basic.start.main               | 3.6.1                        | 6.6.1                        |
| basic.stc.auto                 | 3.7.3                        | 6.7.3                        |
| basic.stc.dynamic              | 3.7.4                        | 6.7.4                        |
| basic.stc.dynamic.allocation   | 3.7.4.1                      | 6.7.4.1                      |
| basic.stc.dynamic.deallocation | 3.7.4.2                      | 6.7.4.2                      |
| basic.stc.static               | 3.7.1                        | 6.7.1                        |
| basic.stc.thread               | 3.7.2                        | 6.7.2                        |
| basic.types                    | 3.9                          | 6.9                          |

| Stable Name                    | C++14 based numbering | C++17 based numbering |
|--------------------------------|-----------------------|-----------------------|
| class                          | 9                     | 12                    |
| class.access                   | 11                    | 14                    |
| class.copy                     | 12.8                  | 15.8                  |
| class.derived                  | 10                    | 13                    |
| class.virtual                  | 10.3                  | 13.3                  |
| compliance                     | 17.6.1.3              | 20.5.1.3              |
| conv                           | 4                     | 7                     |
| coroutine.handle               | 18.11.2               | 21.11.2               |
| coroutine.handle.compare       | 18.11.2.6             | 21.11.2.6             |
| coroutine.handle.con           | 18.11.2.1             | 21.11.2.1             |
| coroutine.handle.export.import | 18.11.2.2             | 21.11.2.2             |
| coroutine.handle.hash          | 18.11.2.7             | 21.11.2.7             |
| coroutine.handle.observers     | 18.11.2.3             | 21.11.2.3             |
| coroutine.handle.promise       | 18.11.2.5             | 21.11.2.5             |
| coroutine.handle.resumption    | 18.11.2.4             | 21.11.2.4             |
| coroutine.traits               | 18.11.1               | 21.11.1               |
| coroutine.traits.primary       | 18.11.1.1             | 21.11.1.1             |
| coroutine.trivial.awaitables   | 18.11.3               | 21.11.3               |
| cpp                            | 16                    | 19                    |
| cstdint                        | 18.4                  | 21.4                  |
| dcl.constexpr                  | 7.1.5                 | 10.1.5                |
| dcl.dcl                        | 7                     | 10                    |
| dcl.decl                       | 8                     | 11                    |
| dcl.fct.def                    | 8.4                   | 11.4                  |
| dcl.fct.def.coroutine          | 8.4.4                 | 11.4.4                |
| dcl.fct.default                | 8.3.6                 | 11.4.2                |
| dcl.spec                       | 7.1                   | 10.1                  |
| dcl.spec.auto                  | 7.1.6.4               | 10.1.7.4              |

| Stable Name         | C++14 based numbering | C++17 based numbering |
|---------------------|-----------------------|-----------------------|
| except              | 15                    | 18                    |
| except.throw        | 15.1                  | 18.1                  |
| expr                | 5                     | 8                     |
| expr.ass            | 5.17                  | 8.18                  |
| expr.await          | 5.3.8                 | 8.3.8                 |
| expr.call           | 5.2.2                 | 8.2.2                 |
| expr.const          | 5.19                  | 8.2                   |
| expr.log.or         | 5.15                  | 8.15                  |
| expr.new            | 5.3.4                 | 8.3.4                 |
| expr.typeid         | 5.2.8                 | 8.2.8                 |
| expr.unary          | 5.3                   | 8.3                   |
| expr.unary.noexcept | 5.3.7                 | 8.3.7                 |
| expr.yield          | 5.20                  | `8.20                 |
| intro               | 1                     | 4                     |
| intro.compliance    | 1.4                   | 4.1                   |
| intro.defs          | 1.3                   | 3                     |
| intro.execution     | 1.9                   | 4.6                   |
| language.support    | 18                    | 21                    |
| lex                 | 2                     | 5                     |
| lex.key             | 2.12                  | 5.11                  |
| library             | 17                    | 20                    |
| meta                | 20.1                  | 23.15                 |
| over                | 13                    | 16                    |
| over.match          | 13.3                  | 16.3                  |
| over.match.funcs    | 13.3.1                | 16.3.1                |
| over.match.oper     | 13.3.1.2              | 16.3.1.2              |
| over.oper           | 13.5                  | 16.5                  |
| special             | 12                    | 15                    |

| <b>Stable Name</b>       | <b>C++14 based numbering</b> | <b>C++17 based numbering</b> |
|--------------------------|------------------------------|------------------------------|
| stmt.iter                | 6.5                          | 9.5                          |
| stmt.jump                | 6.6                          | 9.6                          |
| stmt.ranged              | 6.5.4                        | 9.5.4                        |
| stmt.return              | 6.6.3                        | 9.6.3                        |
| stmt.return.coroutine    | 6.6.3.1                      | 9.6.3.1                      |
| stmt.stmt                | 6                            | 9                            |
| support.coroutine        | 18.11                        | 21.11                        |
| support.dynamic          | 18.6                         | 21.6                         |
| support.exception        | 18.8                         | 21.8                         |
| support.general          | 18.1                         | 21.1                         |
| support.initlist         | 18.9                         | 21.9                         |
| support.limits           | 18.3                         | 21.3                         |
| support.rtti             | 18.7                         | 21.7                         |
| support.runtime          | 18.1                         | 21.10                        |
| support.start.term       | 18.5                         | 21.5                         |
| support.types            | 18.2                         | 21.2                         |
| cpp.headers.freestanding | Table 16                     | Table 19                     |
| lang.sup.lib.summary     | Table 29                     | Table 32                     |
| temp                     | 14                           | 17                           |
| temp.deduct              | 14.8.2                       | 17.8.2                       |
| temp.type                | 14.4                         | 17.4                         |
| unord.hash               | 20.9.12                      | 23.14.15                     |

## 1. Scope

Replace reference to ISO/IEC 14882:2014 with ISO/IEC 14882:2017.

## 2. Normative References

Replace reference to ISO/IEC 14882:2014 with ISO/IEC 14882:2017.

### 3. Terms and Definitions

No changes.

### 4. General

#### 4.3 Program execution

Replace references to paragraph 6 to paragraph 7 to match C++17.

#### 4.4 Lexical conventions [lex]

Move subclause 4.4 into a subclause 5.11 of clause 5.

#### 4.5 Basic concepts [basic]

Move subclause 4.5 into a subclause 6.6.1 of clause 6.

#### 4.6 Dynamic storage duration [basic.stc.dynamic]

Move subclause 4.6 into a subclause 6.7.4 of clause 6.

### 5 Lexical conventions [lex]

#### 5.11 Keywords [lex.key]

[Editorial note: This is the text moved from section 4.4 of the Coroutines TS with references to clauses, paragraphs and tables updated to match C++17].

In subclause 5.11 of the C++ Standard add the keywords `co_await`, `co_yield`, and `co_return` to Table 5 "Keywords".

### 6 Basic Concepts [basic]

#### 6.6.1 `main` function

[Editorial note: This is the text moved from section 4.5 of the Coroutines TS with references to clauses, paragraphs and tables updated to match C++17].

In subclause 6.6.1 of the C++ Standard add underlined text to paragraph 3.

3 The function `main` shall not be used within a program. The linkage (6.5) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed. The function `main` shall not be a coroutine (11.4.4) ...

#### 6.7.4 Dynamic Storage Duration [basic.stc.dynamic]

[Editorial note: This is the text moved from section 4.6 of the Coroutines TS with references to clauses, paragraphs and tables updated to match C++17].

In subclause 6.7.4.1 of the C++ Standard add underlined text to paragraph 4.

4 A global allocation function is only called as the result of a new expression (8.3.4), or called directly using the function call syntax (8.2.2), or called indirectly to allocate storage for a coroutine frame (11.4.4), ...

## 7 Standard Conversions [conv]

No changes are made to Clause 7 of the C++ Standard

## 8 Expressions [expr]

[Editorial note: Only clause and subclause number updates in this clause of the TS].

## 9 Statements [stmt.stmt]

### 9.5.4 The range-based for statement [stmt.ranged]

Update paragraph 1 of subclause 9.5.4 of the TS to match the structure of the same paragraph in the C++17 standard.

1 The range-based for statement

for co\_await<sub>opt</sub> (*for-range-declaration* : *for-range-initializer*) *statement*

is equivalent to

```
{
    auto &&__range = for-range-initializer;
    auto __begin = co_awaitopt begin-expr;
    auto __end = end-expr;
    for (; __begin != __end; co_awaitopt ++__begin) {
        for-range-declaration = *__begin;
        statement
    }
}
```

where

(1.1) — co\_await is present if and only if it appears immediately after the for keyword;

(1.2) ... [Editorial note: with remaining bullets appropriately renumbered].

### 9.6.3.1 The co\_return statement [stmt.return.coroutine]

Update *coroutine-return-statement* production in section 9.6.3.1 of the TS as follows:

*coroutine-return-statement*:

~~co\_return expression<sub>opt</sub>~~

~~co\_return braced-init-list~~

co\_return *expr-or-braced-init-list*<sub>opt</sub>

Update paragraph 2 in section 9.6.3.1 of the TS as follows:

2 The ~~expression or braced-init-list~~ *expr-or-braced-init-list* of a co\_return statement is called its operand.

...

where *final\_suspend* is as defined in 11.4.4 and *S* is defined as follows:

~~(2.1) — *S* is p.return\_value(braced-init-list), if the operand is a braced-init-list;~~

~~(2.2) — *S* is p.return\_value(expression), if the operand is an expression of non-void type;~~

(2.1) - *S* is p.return\_value(*expr-or-braced-init-list*), if the operand is a *braced-init-list* or an expression of non-void type;

(2.2) *S* is { *expression*<sub>opt</sub>; p.return\_void(); }, otherwise;

## 10 Declarations [dcl.dcl]

[Editorial note: Only clause and subclause number updates in this clause of the TS].

## 11 Declarators [dcl.decl]

[Editorial note: Only clause and subclause number updates in this clause of the TS].

## 12 Classes [class]

No changes are made to this Clause of the C++ Standard

## 13 Derived classes [class.derived]

No changes are made to this Clause of the C++ Standard

## 14 Derived classes [class.derived]

No changes are made to this Clause of the C++ Standard

## 15 Special member functions [special]

Replace reference to paragraph 31 of subclause of 12.8 [class.copy/C++14] with a reference to paragraph 1 of 15.8.3 [class.copy.elision].

Add new ~~paragraph after paragraph 31~~ bullet to paragraph 1.

(1.4) in a coroutine (11.4.4), a copy of a coroutine parameter can be omitted and references to that copy replaced with references to the corresponding parameter if the meaning of the program will be unchanged except for the execution of a constructor and destructor for the parameter copy object

Replace reference to paragraph 33 of subclause of 12.8 [class.copy/C++14] with a reference to paragraph 3 of 15.8.3 [class.copy.elision].

Modify paragraph ~~33~~ as follows:

3 In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

(3.1) — If the expression in a return statement (9.6.3) or co return (9.6.3.1) is a (possibly parenthesized) id-expression that names an object with automatic storage duration declared in the body or parameter-declaration-clause of the innermost enclosing function or lambda-expression, or

(3.2) — if the operand of a throw-expression (8.17) is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one),

overload resolution to select the constructor for the copy or the return value overload to call is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, or if the type of the first parameter of the selected constructor or return value overload is not an rvalue reference to the object's type (possibly cv-qualified), overload resolution is performed again, considering the object as an lvalue. [ Note: This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor or return value overload to be called if elision is not performed, and the selected constructor or return value overload must be accessible even if the call is elided. —end note ]

## 16 Overloading [over]

[Editorial note: Only clause and subclause number updates in this clause of the TS].

## 17 Templates [temp]

No changes are made to this Clause of the C++ Standard

## 18 Exception handling [except]

No changes are made to this Clause of the C++ Standard

## 19 Preprocessing directives [cpp]

No changes are made to this Clause of the C++ Standard

## 20 Library Introduction [library]

[Editorial note: Only clause, table number and subclause number updates in this clause of the TS].

## 21 Language support library [language.support]

### 21.11.2.7 Hash support [coroutine.handle.hash]

Update paragraph 1 to match the style used for hash specializations in C++17.

```
template <class P> struct hash<experimental::coroutine_handle<P>>;
```

~~1 The template specialization shall meet the requirements of class template hash (20.9.12).~~

1 The specialization is enabled (23.14.15).